Chapter 5

# Dependent local tasks with collaborative actions

Robots are deployed to move around within the workspace and more importantly to perform various actions to interact with it. Thus in this chapter, we first introduce a new method to construct the model of both robot motion and actions. Based on this model, we can specify a local temporal task as the desired robot motion and actions. Then a planning scheme for both robot motion and actions is proposed as an extension to the nominal solution presented in Chapter 3. Furthermore, inter-robot collaborations are essential for many multi-robot applications to improve system capability, efficiency and robustness. Thus we start from defining a dependency relation among the robots using collaborative and assisting actions. Then a distributed and on-line coordination scheme is proposed where each robot sends requests to its neighbors for collaborative tasks, confirms others' requests and adapts its local plan to fulfill the confirmed collaboration. All decisions are made locally by each robot based on local computation and communication between neighboring robots. This scheme is scalable and resilient to robot failures as the dependency relation is formed and removed dynamically according to the plan execution status and robot capabilities, instead of pre-assigned robot identities. Numerical simulations are provided in the end to validate the proposed scheme.

## 5.1 Motion and action planning

The task specifications considered so far only involve a sequence of regions to visit. However, for practical problems it is often necessary to perform various actions at different regions. In other words, the purpose of "going somewhere" is to "do something". The resulting plan is clearly a combination of transitions among different places and performing sequential actions. It would be inadequate to carry out the motion planning and action planning independently since the motion plan and action plan are closely related, i.e., "where to go" is motivated by "what to do there" and "what to do now" depends on "where it has been". Another observation

is that some actions can only be performed when certain conditions are fulfilled and as a result certain state variables might be changed. Moreover, we propose to separate the domain-specific knowledge [146] such as the workspace model and the robot's mobility in the workspace, from the domain-independent knowledge such as the action map based on the actions the robot is capable of. One advantage is the increased modularity that our framework is adaptable whenever the workspace is modified or the task specification is changed.

### 5.1.1 Complete robot model

The complete robot model is derived by composing the abstraction of robot mobility and the model of robot actions.

#### Abstraction of robot motion

Firstly, in order to distinguish the FTSs for mobility and action, we replace the FTS $\mathcal{T}_c$ in Definition 3.3 with $\mathcal{M}$ as the abstraction of robot's *mobility*:

$$\mathcal{M} = (\Pi_{\mathcal{M}}, \sigma_{\mathcal{M}}, \longrightarrow_{\mathcal{M}}, \Pi_{\mathcal{M},0}, \Psi_{\mathcal{M}}, L_{\mathcal{M}}, W_{\mathcal{M}}), \tag{5.1}$$

of which the notations are similar to $\mathcal{T}_c$; $\sigma_{\mathcal{M}}$ stands for the control strategy (3.3) symbolically; $\Psi_{\mathcal{M}} = \Psi_r \cup \Psi_p$ where $\Psi_r = \{\Psi_{r,i}\}$, $i = 1, \cdots, N$, is the set of propositions as region names; $\Psi_p = \{\Psi_{p,1}, \cdots, \Psi_{p,I}\}$ is the finite set of propositions as interested properties, some of which are relevant to robot's actions discussed later. For instance, "this room has product $\mathsf{A}$" is a property relevant to the action "pickup $\mathsf{A}$".

#### Model of robot actions

Action description language [42] provides an intuitive and powerful way for describing the preconditions and effects of different actions. Classic planning formalisms, like STRIPS [37, 119] provide an intuitive way to describe high-level actions the robot is capable of. Given a set of system states and actions, each action is described by specifying its precondition and effect on the states. Assume that the robot is capable of performing $K$ different actions $\{\sigma_{\mathcal{B},1}, \cdots, \sigma_{\mathcal{B},K}\}$, implementable by the corresponding low-level controllers $\{\mathcal{K}_k\}$, $k = 1, \cdots, K$. Denote by $\Sigma_{\mathcal{B}} = \{\sigma_{\mathcal{B},0}, \cdots, \sigma_{\mathcal{B},K}\}$, where $\sigma_{\mathcal{B},0} \triangleq \texttt{None}$ indicates that none of these $K$ actions is performed. Moreover, we introduce another two sets of propositions: (i) $\Psi_s = \{\Psi_{s,j}\}$, represents the internal states of the robot, $j = 1, 2 \cdots, J$, e.g., "the robot has product $\mathsf{A}$"; (ii) $\Psi_b = \{\Psi_{b,k}\}$, where $\Psi_{b,k} = \top$ if and only if action $k$ is performed, $k = 0, 1, \cdots, K$. We assume that any two actions cannot be concurrent, i.e., at most one element of $\Psi_b$ can be true. The subscripts of $\Psi_s$ and $\Psi_b$ stand for the "state" and "behavior". Then each action in $\Sigma_{\mathcal{B}}$ is described by its precondition and effect functions:

The **precondition function**:

$$\mathsf{Cond} : \Sigma_{\mathcal{B}} \times 2^{\Psi_p} \times 2^{\Psi_s} \longrightarrow \mathsf{True/False}, \tag{5.2}$$

takes one action in $\Sigma_{\mathcal{B}}$, subsets of $\Psi_p$ and $\Psi_s$ as inputs and returns a boolean value. Namely in order to perform that action, the conditions on the workspace properties and the robot's internal states have to be fulfilled. For instance, the action "pickup A" can only be performed if "the room has product A". While some actions like "take pictures" might be performed without such constraints and the condition is $\top$. Note the condition for $\sigma_{\mathcal{B},0}$ is $\top$ by definition.

The **effect function**:

$$\texttt{Eff} : \Sigma_{\mathcal{B}} \times (2^{\Psi_s} \times \Psi_b) \longrightarrow (2^{\Psi_s} \times \Psi_b), \tag{5.3}$$

represents the effect of the actions. As a result of performing action $\sigma_{\mathcal{B},k}$, the robot's internal states $\Psi_s$ might be changed and $\Psi_b$ is changed to indicate which action is performed. More specifically, (i) $\texttt{Eff}(\sigma_{\mathcal{B},0}, w_s, \Psi_{b,k}) = (w_s, \Psi_{b,0})$, where $w_s \subseteq 2^{\Psi_s}$ and $\forall \Psi_{b,k} \in \Psi_b$. Performing $\sigma_{\mathcal{B},0}$ does not change the robot's internal state and all elements in $\Psi_b$ except $\Psi_{b,0}$ are set to $\perp$; (ii) $\texttt{Eff}(\sigma_{\mathcal{B},k}, w_s, \Psi_{b,l}) = (w'_s, \Psi_{b,k})$, where $w_s, w'_s \subseteq 2^{\Psi_s}$ and $\Psi_{b,l}, \Psi_{b,k} \in \Psi_b$, is the effect function of $\sigma_{\mathcal{B},k}$ for $k \neq 0$. For example, once the action "pickup A" is performed, the propositions "the robot has A" and "'pickup A' is performed" become true. Note that the effect functions can not modify the properties of the workspace.

Given $\Psi_p$, $\Psi_s$, $\Psi_b$ and $\Sigma_{\mathcal{B}}$, $\texttt{Cond}$, $\texttt{Eff}$, the *action map* is defined as a tuple

$$\mathcal{B} = (\Pi_{\mathcal{B}}, \Sigma_{\mathcal{B}}, \Psi_p, \hookrightarrow_{\mathcal{B}}, \Pi_{\mathcal{B},0}, \Psi_{\mathcal{B}}, L_{\mathcal{B}}, W_{\mathcal{B}}), \tag{5.4}$$

where (i) $\Pi_{\mathcal{B}} \subseteq 2^{\Psi_s} \times \Psi_b$ is set of all assignments of $\Psi_s$ and $\Psi_b$; (ii) $\Psi_p$ serves as the input propositions, and $2^{\Psi_p}$ is the finite set of possible input assignments; (iii) the conditional transition relation $\hookrightarrow_{\mathcal{B}}$ is defined by $\pi_{\mathcal{B}} \times \alpha_{\mathcal{B}} \times 2^{\Psi_p} \times \pi'_{\mathcal{B}} \subseteq \hookrightarrow_{\mathcal{B}}$ if the following conditions hold: (1) $\alpha_{\mathcal{B}} \in \Sigma_{\mathcal{B}}$, $\pi_{\mathcal{B}}, \pi'_{\mathcal{B}} \in \Pi_{\mathcal{B}}$; (2) $\texttt{Cond}(\alpha_{\mathcal{B}}, 2^{\Psi_p}, \pi_{\mathcal{B}}) = \top$; (3) $\pi'_{\mathcal{B}} \in \texttt{Eff}(\alpha_{\mathcal{B}}, \pi_{\mathcal{B}})$; (iv) $\Pi_{\mathcal{B},0} \subseteq 2^{\Psi_s} \times \Psi_{b,0}$ is the initial state; (v) $\Psi_{\mathcal{B}} = \Psi_s \cup \Psi_b$ is the set of atomic propositions; (vi) $L_{\mathcal{B}}(\pi_{\mathcal{B}}) = \{\pi_{\mathcal{B}}\}$, i.e., the labeling function is the state itself; (vii) $W_{\mathcal{B}} : \hookrightarrow_{\mathcal{B}} \to \mathbb{R}^+$ is the weight associated with each transition and $W_{\mathcal{B}}(\pi_{\mathcal{B}}, \alpha_{\mathcal{B}}, 2^{\Psi_p}, \pi'_{\mathcal{B}})$ is estimated by the cost of action $\alpha_{\mathcal{B}}$.

***Remark* 5.1.** The set of states $\Pi_{\mathcal{B}}$ is defined as $2^{\Psi_s} \times \Psi_b$ instead of $2^{\Psi_s} \times 2^{\Psi_b}$ because only one element in $\Psi_b$ can be true. ▲

$\Psi_p$ can be viewed as external inputs [11] to the action map, i.e., within different regions the transition relations might be different due to their different properties. Moreover, $\mathcal{B}$ is nondeterministic in the sense that at each state $\pi_{\mathcal{B}}$ any action whose associated condition function is evaluated to be true, can be performed. It is worth mentioning that the action map is constructed independently of the structure of the workspace where the robot will be deployed. Furthermore, given an instance of the workspace property $\Psi_p$, the action map $\mathcal{B}$ is equivalent to a wFTS as all conditional transition relations can be verified or falsified.

### Complete functionality model

As mentioned earlier, the mobility abstraction $\mathcal{M}$ from (5.1) and the action map $\mathcal{B}$ from (5.4) are adequate for the controller synthesis within certain problem domain.

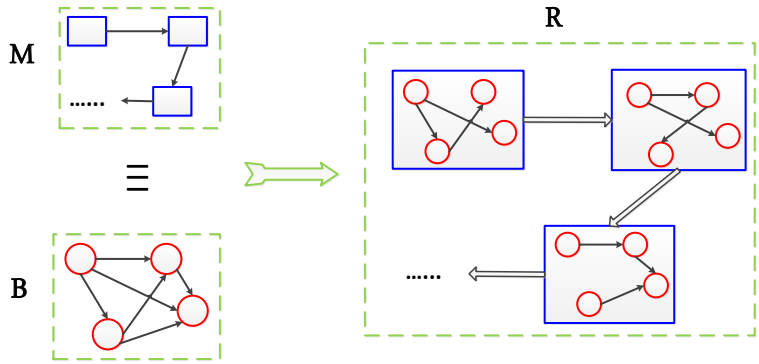


Figure 5.1: The action map $\mathcal{B}$ is composed with each region of $\mathcal{M}$, giving a complete description $\mathcal{R}$ of robot's functionalities.

However, in order to consider richer and more complex tasks involving both robot motion and actions, we need a *complete model* of robot's functionalities that combines these two parts. We propose the following way to compose $\mathcal{M}$ and $\mathcal{B}$ as a FTS:

$$\mathcal{R} = (\Pi_{\mathcal{R}}, \Sigma_{\mathcal{R}}, \longrightarrow_{\mathcal{R}}, \Pi_{\mathcal{R},0}, \Psi_{\mathcal{R}}, L_{\mathcal{R}}, W_{\mathcal{R}}), \tag{5.5}$$

where (i) $\Pi_{\mathcal{R}} = \Pi_{\mathcal{M}} \times \Pi_{\mathcal{B}}$ is the set of states; (ii) $\Sigma_{\mathcal{R}} = \sigma_{\mathcal{M}} \cup \Sigma_{\mathcal{B}}$ is the set of actions; (iii) $\longrightarrow_{\mathcal{R}} \subseteq \Pi_{\mathcal{R}} \times \Sigma_{\mathcal{R}} \times \Pi_{\mathcal{R}}$ is the transition relation, defined by the following rules: (1) $\langle \pi_{\mathcal{M}}, \pi_{\mathcal{B}} \rangle \xrightarrow{\sigma_{\mathcal{M}}}_{\mathcal{R}} \langle \pi'_{\mathcal{M}}, \pi'_{\mathcal{B}} \rangle$ if $\pi_{\mathcal{M}} \xrightarrow{\sigma_{\mathcal{M}}}_{\mathcal{M}} \pi'_{\mathcal{M}}$ and $\pi_{\mathcal{B}} \xrightarrow{\sigma_{\mathcal{B},0}}_{\mathcal{B}} \pi'_{\mathcal{B}}$; (2) $\langle \pi_{\mathcal{M}}, \pi_{\mathcal{B}} \rangle \xrightarrow{\alpha_{\mathcal{B}}}_{\mathcal{R}} \langle \pi_{\mathcal{M}}, \pi'_{\mathcal{B}} \rangle$ if $\pi_{\mathcal{B}} \times \alpha_{\mathcal{B}} \times L_{\mathcal{M}}(\pi_{\mathcal{M}}) \times \pi'_{\mathcal{B}} \subset \hookrightarrow_{\mathcal{B}}$, where $\alpha_{\mathcal{B}} \in \Sigma_{\mathcal{B}}$. (iv) $\Pi_{\mathcal{R},0} = \Pi_{\mathcal{M},0} \times \Pi_{\mathcal{B},0}$ contains the robot's initial region and initial internal state; (v) $\Psi_{\mathcal{R}} = \Psi_{\mathcal{M}} \cup \Psi_{\mathcal{B}}$ is the complete set of atomic propositions including $\Psi_r$, $\Psi_p$, $\Psi_s$ and $\Psi_b$; (vi) $L_{\mathcal{R}} : \Pi_{\mathcal{R}} \to 2^{\Psi_{\mathcal{R}}}$ is the labeling function, $L_{\mathcal{R}}(\langle \pi_{\mathcal{M}}, \pi_{\mathcal{B}} \rangle) = L_{\mathcal{M}}(\pi_{\mathcal{M}}) \cup L_{\mathcal{B}}(\pi_{\mathcal{B}})$; (vii) $W_{\mathcal{R}} : \longrightarrow_{\mathcal{R}} \to \mathbb{R}^+$, is the weight function on each transition, defined as: (1) $W_{\mathcal{R}}(\langle \pi_{\mathcal{M}}, \pi_{\mathcal{B}} \rangle, \sigma_{\mathcal{M}}, \langle \pi'_{\mathcal{M}}, \pi'_{\mathcal{B}} \rangle) = W_{\mathcal{M}}(\pi_{\mathcal{M}}, \sigma_{\mathcal{M}}, \pi'_{\mathcal{M}})$; (2) $W_{\mathcal{R}}(\langle \pi_{\mathcal{M}}, \pi_{\mathcal{B}} \rangle, \alpha_{\mathcal{R}}, \langle \pi_{\mathcal{M}}, \pi'_{\mathcal{B}} \rangle) = W_{\mathcal{B}}(\pi_{\mathcal{B}}, \alpha_{\mathcal{R}}, \pi'_{\mathcal{B}})$, if $\alpha_{\mathcal{R}} \in \Sigma_{\mathcal{B}}$.

***Remark* 5.2.** Note that $\hookrightarrow_{\mathcal{B}}$, $\sigma_{\mathcal{B},0}$ is released automatically whenever the controller (3.3) is activated. Because whenever the robot moves to a new region, it indicates that no actions within $\sigma_{\mathcal{B},k}$ are performed as we assume non-concurrent actions. ▲

Figure 5.1 illustrates the idea behind the process of parallel composition defined above. Blue squares represent the states of $\mathcal{M}$ and red cycles encode the states of $\mathcal{B}$. Loosely speaking, when composing them into $\mathcal{R}$, $N$ copies of $\mathcal{B}$ are first made, corresponding to the $N$ regions within the workspace. At the same time, the conditional transition relations in these copies are verified or falsified by verifying the conditions on the properties of each region.

### 5.1.2 Local task for motion and actions

The composed system $\mathcal{R}$ is a wFTS over the set of atomic propositions $\Psi_{\mathcal{R}}$. Recall that $\Psi_{\mathcal{R}} = \Psi_r \cup \Psi_p \cup \Psi_s \cup \Psi_b$. Among them, $\Psi_r$, $\Psi_p$ are commonly seen in related work [15, 85, 132], but $\Psi_s$, $\Psi_b$ allow us to express richer requirements on the robot's internal states and actions directly, as for example where these actions are desired and the preferred sequence. For instance, the task "eventually always drop A at region 1" can be expressed as $\varphi = \Box \Diamond (\Psi_{r,1} \wedge \Psi_{b,2})$, where $\Psi_{r,1}$="{the robot is in region 1"} and $\Psi_{b,2}$={"drop A" is performed}. Notice that we do not even need to specify where the robot should perform "pickup A".

### 5.1.3 Hybrid control strategy

We now state the problem we consider in this section:

**Problem 5.1.** *Given the full robot functionality $\mathcal{R}$ and the task $\varphi$, find its motion and action plan that* ***minimizes*** *the cost defined by* (3.9) *and construct the hybrid control strategy that executes this plan.* ▲

Since $\mathcal{R}$ is a standard finite transition system like $\mathcal{T}_c$ from Definition 3.3, the framework proposed in Chapter 3 can be applied directly to find the infinite optimal motion and action sequence that has a finite representation and minimizes the cost by 3.9. Namely, denote by $R_{\text{opt}}$ the optimal accepting run of the product $\mathcal{R} \times \mathcal{A}_\varphi$ from Algorithm 3.1, which can be projected onto $\mathcal{R}$ as the *motion and action* plan $\tau_{\text{opt}} = R_{\text{opt}}|_{\Pi_{\mathcal{R}}}$. For each pair of sequential states $(\pi_{\mathcal{R},s}, \pi_{\mathcal{R},s+1})$ in $\tau_{\text{opt}}$ there exists an action $\alpha_{\mathcal{R}} \in \Sigma_{\mathcal{R}}$ such that $(\pi_{\mathcal{R},s}, \alpha_{\mathcal{R}}, \pi_{\mathcal{R},s+1}) \in \longrightarrow_{\mathcal{R}}$ from (5.5). Thus the underlying low-level control strategy can be synthesized by sequentially implementing the continuous controller associated with the actions along $\tau_{\mathcal{R}}$, in a similar way as Algorithm 3.2. In particular, if $\alpha_{\mathcal{R}} = \sigma_{\mathcal{B},k}$, the controller $\{\mathcal{K}_k\}$ that implements the action $\sigma_{\mathcal{B},k}$ is activated. If $\alpha_{\mathcal{R}} = \sigma_{\mathcal{M}}$, the navigation controller (3.3) is applied to drive the robot from the starting region to one point in the goal region.

The overall framework has four steps: (i) construct the mobility abstraction $\mathcal{M}$ and action map $\mathcal{B}$; (ii) construct the full functionality $\mathcal{R}$ by composing $\mathcal{M}$ and $\mathcal{B}$; (iii) synthesis optimal the motion and action plan $\tau_{\text{opt}}$; (iv) execute the plan by the hybrid control strategy.

It is worth mentioning that $\mathcal{M}$ and $\mathcal{B}$ are constructed only once for the robot within a certain workspace and $\varphi$ can express any task specification in terms of required motions and actions. Steps (ii)-(iv) are performed in an automated way. Whenever a new task specification is given, the complete functionalities model $\mathcal{R}$ remains unchanged and steps (iii)-(iv) are repeated to synthesize the corresponding plan. Whenever the workspace is modified, only $\mathcal{M}$ needs to be re-constructed but the action map $\mathcal{B}$ remains the same and can be reused.

## 5.2 Multi-robot systems with dependent local tasks

We have discussed in the previous section about how to construct a motion and action plan for a single robot when its individual local task can be fulfilled by itself. But what if one robot needs other robots' collaboration to accomplish one action in its plan, namely whether one robot can accomplish its local task depends on other robots' collaboration. In this section, we address this issue by considering a multi-robot system of $N$ robots with identities $i \in \mathcal{N} = \{1, 2, \cdots, N\}$, as described in Section 4.4, with heterogeneous capabilities and dependent local tasks.

### 5.2.1 Collaborative actions

In this part, we construct the complete robot model for both motion and actions, where the abstraction for robot motion is similar to Section 5.1.1 but the model of robot actions is different, due to the existence of collaborative and assisting actions.

Same as before, the workspace consists of $M$ partitions as the regions of interest, denoted by $\Pi = \{\pi_1, \pi_2, \cdots, \pi_M\}$. We assume that these symbols are assigned a priori and the workspace is fully-known by all robots. Robot $i$'s motion within the workspace is modeled as a wFTS:

$$\mathcal{M}_i \triangleq (\Pi, \longrightarrow^i_{\mathcal{M}}, \Pi^i_{\mathcal{M},0}, \Psi^i_{\mathcal{M}}, L^i_{\mathcal{M}}, T^i_{\mathcal{M}}), \tag{5.6}$$

where $\longrightarrow^i_{\mathcal{M}} \subseteq \Pi \times \Pi$ is the transition relation; $\Pi^i_{\mathcal{M},0} \in \Pi$ is the initial region robot $k$ starts from; $L^i_{\mathcal{M}} : \Pi \to 2^{\Psi^i_{\mathcal{M}}}$ is the labeling function, indicating the properties held by each region; $T^i_{\mathcal{M}} : \longrightarrow^i_{\mathcal{M}} \to \mathbb{R}^+$ estimates the time each transition takes. A path of $\mathcal{M}_i$ is a sequence of regions $\pi_0\pi_1\cdots\pi_P$, where $(\pi_n, \pi_{n+1}) \in \longrightarrow^i_{\mathcal{M}}$, $\forall n = 0, 1, \cdots, P-1$. Note that $\mathcal{M}_i$ might be different between the robots due to heterogeneity. Moreover, each robot $i$ has a set of neighboring robots, denoted by $\mathcal{N}_i \subseteq \mathcal{N}$ and robot $i$ can exchange messages directly with any robot $j \in \mathcal{N}_i$.

#### Action model

Besides the motion ability, robot $i$ is capable of performing a set of actions denoted by $\Sigma_i \triangleq \Sigma^i_l \cup \Sigma^i_c \cup \Sigma^i_h$, where

- $\Sigma^i_l$ is a set of *local* actions, which can be done by robot $i$ itself;
- $\Sigma^i_c$ is a set of *collaborative* actions, which can be done by robot $i$ but requires collaborations from other robots;
- $\Sigma^i_h$ is a set of *assisting* actions, which robot $i$ offers to other robots to accomplish their collaborative actions.

In other words, $\Sigma^i_l$ and $\Sigma^i_c$ contain active actions that can be *initiated* by robot $i$, denoted by $\Sigma^i_a = \Sigma^i_l \cup \Sigma^i_c$, while $\Sigma^i_h$ contains assisting actions only to *assist* other robots. By default, $\sigma_0 = \texttt{None} \in \Sigma^i_l$ means that none of the actions is performed.

Moreover, denote by $\Sigma_h^{\sim i}$ the set of *external* assisting actions robot $i$ depends on, which can only be provided by some of its neighbors in $\mathcal{N}_i$, i.e., $\Sigma_h^{\sim i} \subseteq \cup_{i \in \mathcal{N}_i} \Sigma_h^i$. Table 5.1 shows the action sets of the robots that will be simulated in Section 5.3. The action model of robot $i$ is modeled by a six-tuple:

$$\mathcal{B}_i \triangleq (\Sigma_i, \Psi_\Sigma^i, L_\Sigma^i, \texttt{Cond}_i, \texttt{Dura}_i, \texttt{Depd}_i), \tag{5.7}$$

where $\Sigma_i$ is the set of actions defined earlier; $\Psi_\Sigma^i$ is a set of atomic propositions related to the robot's *active* actions; $L_\Sigma^i : \Sigma_i \to 2^{\Psi_\Sigma^i}$ is the labeling function. $L_\Sigma^i(\sigma_h) = \varnothing$, $\forall \sigma_h \in \Sigma_h^i$ and $L_\Sigma^i(\sigma_a) \subseteq \Psi_\Sigma^i$, $\forall \sigma_a \in \Sigma_a^i$; $\texttt{Cond}_i : \Sigma_i \times 2^{\Psi_\mathcal{M}^i} \to \top/\bot$ indicates the set of region properties that have to be fulfilled in order to perform an action; $\texttt{Dura}_i : \Sigma_i \to \mathbb{R}^+$ is the time duration of each action. $\texttt{Dura}_i(\sigma_0) = T_0 > 0$ is a design parameter; $\texttt{Depd}_i : \Sigma_i \to 2^{\Sigma_h^{\sim i}}$ is the *dependence* function. $\texttt{Depd}_i(\sigma_s) = \varnothing$, $\forall \sigma_s \in \Sigma_l^i \cup \Sigma_h^i$ and $\texttt{Depd}_i(\sigma_c) \subseteq \Sigma_h^{\sim i}$, $\forall \sigma_c \in \Sigma_c^i$. Namely each collaborative action depends on a set of assisting actions from its neighbors. This is useful for defining complex collaborations involving multiple robots.

***Remark* 5.3.** Compared with defining dependency directly on robot identities, our action model allows more flexibility since their identities need not be known a priori and new or existing robots can be added or removed. Moreover, different from (5.4), the action model by (5.7) can model both local and collaborative actions. ▲

**Definition 5.1.** A local or assisting action $\sigma_s \in \Sigma_l^i \cup \Sigma_h^i$ is **done** at region $\pi_v \in \Pi$ if two conditions hold: (i) $\texttt{Cond}_i(\sigma_s, L_\mathcal{M}^i(\pi_v)) = \top$; (ii) $\sigma_s$ is activated for period $\texttt{Dura}_i(\sigma_s)$. For a collaborative action $\sigma_c \in \Sigma_c^i$, another condition is needed: (iii) all assisting actions in $\texttt{Depd}_i(\sigma_c)$ are done by other robots at the same region $\pi_v$. ■

### Complete robot model

A complete robot model of robot $i$, denoted by $\mathcal{R}_i$, refers to the finite transition system that models both its motion and actions.

**Definition 5.2.** Given $\mathcal{M}_i$ and $\mathcal{B}_i$, robot $i$'s complete model can be constructed as follows: $\mathcal{R}_i = (\Pi_\mathcal{R}^i, \longrightarrow_\mathcal{R}^i, \Pi_{\mathcal{R},0}^i, \Psi_\mathcal{R}^i, L_\mathcal{R}^i, T_\mathcal{R}^i)$, where $\Pi_\mathcal{R}^i = \Pi \times \Sigma_i$. $\pi_{\mathcal{R},s} = \langle \pi_t, \sigma_n \rangle \in \Pi_\mathcal{R}^i$, $\forall \pi_t \in \Pi$, $\forall \sigma_n \in \Sigma_i$; $\longrightarrow_\mathcal{R}^i \subseteq \Pi_\mathcal{R}^i \times \Pi_\mathcal{R}^i$. $\big(\langle \pi_s, \sigma_m \rangle, \langle \pi_t, \sigma_n \rangle\big) \in \longrightarrow_\mathcal{R}^i$ if (i) $\sigma_n = \sigma_m = \sigma_0$, $\pi_s \longrightarrow_\mathcal{M}^i \pi_t$; *or* (ii) $\sigma_m = \sigma_0$, $\sigma_n \neq \sigma_0$ and $\pi_s = \pi_t$, $\texttt{Cond}_i(\sigma_n, L_\mathcal{M}^i(\pi_s)) = \top$; *or* (iii) $\sigma_m \in \Sigma_i$, $\sigma_n = \sigma_0$ and $\pi_s = \pi_t$; $\Pi_{\mathcal{R},0}^i = \Pi_{\mathcal{M},0}^i \times \sigma_0$ is the initial state; $\Psi_\mathcal{R}^i = \Psi_\mathcal{M}^i \cup \Psi_\Sigma^i$; $L_\mathcal{R}^i : \Pi_\mathcal{R}^i \to 2^{\Psi^i}$. $L_\mathcal{R}^i(\langle \pi_s, \sigma_m \rangle) = L_\mathcal{M}^i(\pi_s) \cup L_\Sigma^i(\sigma_m)$; $T_\mathcal{R}^i : \longrightarrow_\mathcal{R}^i \to \mathbb{R}^+$. For case (i) above, $T_\mathcal{R}^i(\langle \pi_s, \sigma_m \rangle, \langle \pi_t, \sigma_n \rangle) = T_\mathcal{M}^i(\pi_s, \pi_t)$; for case (ii), $T_\mathcal{R}^i(\langle \pi_s, \sigma_m \rangle, \langle \pi_t, \sigma_n \rangle) = \texttt{Dura}_i(\sigma_n)$; for case (iii), $T_\mathcal{R}^i(\langle \pi_s, \sigma_m \rangle, \langle \pi_t, \sigma_n \rangle) = T_0$. ■

Note that when defining $\longrightarrow_\mathcal{R}^i$ above, the condition of performing an action is verified over the properties of each region. Thus $\mathcal{R}_i$ is a standard FTS by [11]. Its finite path is denoted by $\tau_i = \pi_{\mathcal{R},0}\pi_{\mathcal{R},1}\cdots\pi_{\mathcal{R},P}$, where $\pi_{\mathcal{R},s} \in \Pi_\mathcal{R}^i$, $\pi_{\mathcal{R},0} \in \Pi_{\mathcal{R},0}^i$ and $(\pi_{\mathcal{R},s}, \pi_{\mathcal{R},s+1}) \in \longrightarrow_\mathcal{R}^i$, $\forall s = 0, \cdots, P-1$, of which the trace is given by $\texttt{trace}(\tau_i) = L_\mathcal{R}^i(\pi_{\mathcal{R},0}) L_\mathcal{R}^i(\pi_{\mathcal{R},1}) \cdots L_\mathcal{R}^i(\pi_{\mathcal{R},P})$.

| **Robot** | $\Sigma_l^i$ | $\Sigma_c^i$ | $\Sigma_h^i$ | $\Sigma_h^{\sim i}$ |
|---|---|---|---|---|
| $\mathfrak{R}_1$ | $l_A, u_A$ | $l_B, u_B$ | $\varnothing$ | $h_B$ |
| $\mathfrak{R}_2$ | $s$ | $\varnothing$ | $h_B, h_{C_1}, h_F$ | $\varnothing$ |
| $\mathfrak{R}_3$ | $\varnothing$ | $o_M$ | $h_{C_2}, h_F$ | $h_M$ |
| $\mathfrak{R}_4$ | $s$ | $a_C$ | $h_M, h_F$ | $h_{C_1}, h_{C_2}$ |
| $\mathfrak{R}_5$ | $m_D$ | $\varnothing$ | $h_B, h_{C_1}, h_{C_2}$ | $\varnothing$ |
| $\mathfrak{R}_6$ | $o_E$ | $c_F$ | $h_B, h_M$ | $h_F$ |

Table 5.1: Action sets (except $\sigma_0$) described in Section 5.3.

### 5.2.2 Problem formulation

The local task of robot $i$, denoted by $\varphi_i$, is given as an sc-safe LTL formula over the set of atomic propositions $\Psi_{\mathcal{R}}^i$ from Definition 5.2. Thus $\varphi_i$ can contain requirements on robot's motion, local and collaborative actions. The syntax and semantics of sc-LTL can be found in Section 3.2. As also mentioned there, an sc-safe LTL formula can be fulfilled by a finite prefix. In particular, given a finite path $\tau_i$ of $\mathcal{R}_i$, then $\tau_i$ fulfils $\varphi_i$ if $\texttt{trace}(\tau_i) \vDash \varphi_i$ where the satisfaction relation is defined there. One special case is that when $\varphi_i \triangleq \top$, robot $i$ does not have a local task and serves as an assisting robot. In summary, we address the following problem in this section:

**Problem 5.2.** *Given $\mathcal{R}_i$ and its locally-assigned task $\varphi_i$ that depends on other robots' collaboration, design a distributed control and coordination scheme such that $\varphi_i$ is fulfilled for all $i \in \mathcal{N}$.* ▲

### 5.2.3 Distributed task coordination

This section contains the main contribution as the bottom-up motion and task coordination scheme for multi-robot systems with dependent local tasks. It includes the off-line initial plan synthesis, the on-line request and reply messages exchange protocol, the real-time plan adaptation algorithm and the failure recovery mechanism.

#### Initial plan synthesis

Given the complete robot model and its local task, we can first synthesize an *initial* motion and action plan for each robot, which happens off-line and serves as a starting point for the real-time coordination and adaptation scheme in Section 5.2.3. We intend to find a finite path of $\mathcal{R}_i$ whose trace satisfies the co-safe formula $\varphi_i$ as described in Section 5.2.2. Let $\mathcal{A}_{\varphi_i}$ be the NBA associated with $\varphi_i$ from Section 3.2, i.e., $\mathcal{A}_{\varphi_i} = (Q_i, 2^{\Psi_{\mathcal{G}}^i}, \delta_i, Q_0^i, \mathcal{F}_i)$. The product automaton $\mathcal{A}_{p,i}$ is defined as follows:

$$\mathcal{A}_{p,i} = \mathcal{R}_i \otimes \mathcal{A}_{\varphi_i} = (Q_{p,i}, \delta_{p,i}, Q_{p,i,0}, \mathcal{F}_{p,i}, W_{p,i}), \tag{5.8}$$

where $Q_{p,i} = \Pi_{\mathcal{R}}^i \times Q_i$; $(\langle \pi_{\mathcal{R},s}, q_m \rangle, \langle \pi_{\mathcal{R},k}, q_n \rangle) \in \delta_{p,i}$ if it holds that $\pi_{\mathcal{R},s} \longrightarrow_{\mathcal{R}}^i \pi_{\mathcal{R},k}$ and $(q_m, L_{\mathcal{R}}^i(\pi_{\mathcal{R},s}), q_n) \in \delta_i$ ; $Q_{p,i,0} = \Pi_{\mathcal{R},0}^i \times Q_0^i$ is the set of initial states; $\mathcal{F}_{p,i} = \Pi_{\mathcal{R}}^i \times$

$\mathcal{F}^i$ is the set of accepting states; $W_{p,i} : \delta_{p,i} \times Q_{p,i} \to \mathbb{R}^+$. $W_{p,i}(\langle \pi_{\mathcal{R},s}, q_m \rangle, \langle \pi_{\mathcal{R},k}, q_n \rangle) = T^i_{\mathcal{R}}(\pi_{\mathcal{R},s}, \pi_{\mathcal{R},k})$, where $\langle \pi_{\mathcal{R},k}, q_n \rangle \in \delta_{p,i}(\langle \pi_{\mathcal{R},s}, q_m \rangle)$.

There exists a finite path of $\mathcal{R}_i$ satisfying $\varphi_i$ if and only if $\mathcal{A}_{p,i}$ has a finite path from an initial state to an accepting state. Then this path could be projected back to $\mathcal{R}_i$ as a finite path, the trace of which should satisfy $\varphi_i$ automatically [11]. Let $R^i_p = q^i_{p,0} q^i_{p,1} \cdots q^i_{p,P}$ be a finite path of $\mathcal{A}_{p,i}$, where $q^i_{p,0} \in Q_{p,i,0}$, $q^i_{p,P} \in \mathcal{F}_{p,i}$, $q^i_{p,l} \in Q_{p,i}$ and $(q^i_{p,l}, q^i_{p,l+1}) \in \delta_{p,i}$, $\forall l = 0, 1, \cdots, P-1$. The *cost* of $R^i_p$ is defined by $\texttt{Cost}(R^i_p, \mathcal{A}_{p,i}) = \sum_{l=0}^{P-1} W_{p,i}(q^i_{p,l}, q^i_{p,l+1})$, which is the summed weights along $R^i_p$. The $l_{\text{th}}$ element is given by $R^i_p[l] = q^i_{p,l}$ and the segment from the $l_{\text{th}}$ to the $k_{\text{th}}$ element is $R^i_p[l:k] = q^i_{p,l} q^i_{p,l+1} \cdots q^i_{p,k}$, where $l \leq k \leq P$.

**Problem 5.3.** *Find a finite path $R^i_p$ of $\mathcal{A}_{p,i}$ with the above structure that minimizes its total cost.* ▲

Denote by $R^i_{p,\text{init}}$ the solution to the above problem. Algorithm 1 in [58] solves the above problem, which is also restated in Algorithm 3.1. It utilizes Dijkstra's algorithm for computing the shortest path from any initial state in $Q_{p,i,0}$ to every reachable accepting state in $\mathcal{F}^i_p$ and checks if there is cycle back to $q_{p,P}$. The worst-case complexity [89, 97] is $\mathcal{O}(|\delta_{p,i}| \cdot \log |Q_{p,i}| \cdot |Q_{p,i,0}|)$. By projecting $R^i_{p,\text{init}}$ onto $\Pi^i_{\mathcal{R}}$, it gives the initial motion and action plan $\tau^i_{\mathcal{R},\text{init}} = R^i_{p,\text{init}}|_{\Pi^i_{\mathcal{R}}}$ that fulfils $\varphi_i$.

***Remark* 5.4.** The initial plans are synthesized locally by each robot instead of by a central unit [18] or within a cluster [51]. ▲

The plan $\tau^i_{\mathcal{R},\text{init}}$ can be executed by activating the motion or actions in sequence. However since $\tau^i_{\mathcal{R},\text{init}}$ may contain several collaborative actions from $\Sigma^i_c$ to satisfy $\varphi_i$, the successful execution of $\tau^i_{\mathcal{R},\text{init}}$ depends on other robots' collaboration, which however is not guaranteed since $\tau^i_{\mathcal{R},\text{init}}$ is synthesized off-line and locally. We resolve this problem by a real-time coordination and adaptation scheme in the following.

### On-line collaborative task coordination

There is no guarantee that the initial plan $\tau^i_{\mathcal{R},\text{init}}$ can be executed successfully if it contains collaborative actions. In this part, we propose a distributed and on-line coordination scheme which involves four major parts: (i) a request and reply exchange protocol driven by collaborative actions in a finite horizon; (ii) an optimization and confirmation mechanism, by solving a mixed integer program given the replies; (iii) a real-time plan adaptation algorithm given the confirmation; (iv) an robot failure detection and recovery scheme along with the plan execution.

### Planned motion and actions in horizon

Denote by $\pi^i_{\mathcal{R},t} \in \Pi^i_{\mathcal{R}}$ the state of robot $i$ at time $t$. After the system starts, assume $\pi^i_{\mathcal{R},t}$ is the $l_{th}$ element in $\tau^i_{\mathcal{R},\text{init}}$, namely, $\pi^i_{\mathcal{R},t} = \tau^i_{\mathcal{R},\text{init}}[l]$. Each robot $i \in \mathcal{N}$ is given a bounded planning horizon $0 < H_i < \infty$, which is the time ahead robot $i$ checks

**Algorithm 5.7** Plan in horizon and request, `Request`()

**Input**: $\tau^i_{\mathcal{R},\text{init}}$, $\pi^i_{\mathcal{R},t}$, $H_i$
**Output**: $\tau^i_{\mathcal{R},H}$, $\mathbf{Request}^i$
$\tau^i_{\mathcal{R},\text{init}}[l] = \pi^i_{\mathcal{R},t}$, $s = 0$, $T_m = 0$, $\mathbf{Request}^i = \varnothing$
**while** *($T < H_i$) and ($l + s \leq |\tau^i_{\mathcal{R},init}|$)* **do**
  $s = s + 1$, $T_m = T_m + T^i_{\mathcal{R}}\left(\tau^i_{\mathcal{R},\text{init}}[l], \tau^i_{\mathcal{R},\text{init}}[l+s]\right)$
  $\langle \pi_v, \sigma_m \rangle = \tau^i_{\mathcal{R},\text{init}}[l+s]$
  **if** $\mathbf{Request}^i = \varnothing$ *and* $\sigma_m \in \Sigma^i_c$ **then**
    **forall the** $\sigma_d \in \mathtt{Depd}_i(\sigma_m)$ **do**
      add $(\sigma_d, \pi_v, T_m)$ to $\mathbf{Request}^i$
$k = l + s$, $\tau^i_{\mathcal{R},H} = \tau^i_{\mathcal{R},\text{init}}[l:k]$
**return** $\tau^i_{\mathcal{R},H}$, $\mathbf{Request}^i$

its plan. Similar approach can be found in [153] for a single dynamic system. Then the sequence of states robot $i$ is expected to reach within the time $H_i$, denoted by $\tau^i_{\mathcal{R},H}$, is the segment $\tau^i_{\mathcal{R},H} = \tau^i_{\mathcal{R},\text{init}}[l:k]$, where the index $k \geq l$ is the solution to this optimization problem: min $k$, subject to $\sum_{s=l}^{k} T^i_{\mathcal{R}}\left(\tau^i_{\mathcal{R},\text{init}}[s], \tau^i_{\mathcal{R},\text{init}}[s+1]\right) \geq H_i$. It can be solved by iterating through the sequence of $\tau^i_{\mathcal{R},\text{init}}$ and computing the accumulated cost, which is then compared with $H_i$. If it does not have a solution, it means $H_i$ is larger than the total cost of the rest of the plan $\tau^i_{\mathcal{R},\text{init}}[l:]$, then $k = |\tau^i_{\mathcal{R},\text{init}}|$, see Algorithm 5.7. This time horizon avoids coordinating on collaborative actions that will be done within a long time from now.

### Request to neighbors

Given $\tau^i_{\mathcal{R},H}$ as the motion and actions in horizon, robot $i$ needs to check whether it needs others' collaboration within $\tau^i_{\mathcal{R},H}$. This is done by verifying whether a collaborative action needs to be performed to reach the states in $\tau^i_{\mathcal{R},H}$. More specifically, for the *first* state $\langle \pi_v, \sigma_m \rangle \in \tau^i_{\mathcal{R},H}$ satisfying $\sigma_m \in \Sigma^i_c$, robot $i$ needs to *broadcast* a request to all robots within its communication network $\mathcal{N}_i$ regarding this action. This request message has the following format:

$$\mathbf{Request}^i = \{(\sigma_d, \pi_v, T_m), \forall \sigma_d \in \mathtt{Depd}_i(\sigma_m)\}, \tag{5.9}$$

where $\mathtt{Depd}_i(\sigma_m)$ is the set of external assisting actions that $\sigma_m$ depends on by (5.7); $\pi_v \in \Pi$ is the region where $\sigma_m$ will be performed; $T_m \geq 0$ is the time when $\sigma_m$ will be performed from now. Assume that $\langle \pi_v, \sigma_m \rangle$ is the $k_{th}$ element of $\tau^i_{\mathcal{R},H}$. Then $T_m = \sum_{s=l}^{k} T^i_{\mathcal{R}}\left(\tau^i_{\mathcal{R},H}[s], \tau^i_{\mathcal{R},H}[s+1]\right)$, see Algorithm 5.7. Each element $(\sigma_d, \pi_v, T_m) \in \mathbf{Request}^i$ contains the message that "robot $i$ is requesting the assisting action $\sigma_d$ at region $\pi_v$ in the time $T_m$ from now". The request message from robot $i$ to each robot

**Algorithm 5.8** Reply to request by robot $j$, `Reply()`

**Input**: $\mathbf{Request}_j^i$, $\widehat{R}_{p,-}^j$, $\mathcal{A}_{p,j}$, $\overline{T}_j$
**Output**: $\mathbf{Reply}_i^j$, $\widehat{P}$
**forall the** $(\sigma_d, \pi_v, T_m) \in \mathbf{Request}_j^i$ **do**
  **if** $\overline{T}_j$ *is* 0 **then**
    $\left(\widehat{R}_{p,+}^j, b_d^j, t_d^j\right) = \texttt{EvalReq}(\widehat{R}_{p,-}^j, (\pi_v, \sigma_d, T_m), \mathcal{A}_{p,j})$
    **if** $b_d^j$ *is* $\top$ **then**
      $\widehat{P}(\sigma_d) = \widehat{R}_{p,+}^j$, add $(\sigma_d, b_d^j, t_d^j)$ to $\mathbf{Reply}_i^j$
  add $(\sigma_d, \perp, 0)$ to $\mathbf{Reply}_i^j$
**Return** $\mathbf{Reply}_i^j$, $\widehat{P}$

$j \in \mathcal{N}_i$, denoted by $\mathbf{Request}_j^i$, is the same as $\mathbf{Request}^i$, i.e., $\mathbf{Request}_j^i = \mathbf{Request}^i$, $\forall j \in \mathcal{N}_i$.

***Remark* 5.5.** The request message is sent only for the *first* collaborative action in $\tau_{\mathcal{R},H}^i$ within the time horizon $H_i$ (see Algorithm 5.7), as the outcome of this request would greatly affect the second collaborative action in $\tau_{\mathcal{R},H}^i$. ▲

### Request evaluation and reply

Upon receiving the request, robot $j \in \mathcal{N}_i$ needs to evaluate this request in terms of *feasibility* and *cost*, in order to reply to robot $i$. Specifically, the reply message from robot $j$ to robot $i$ has the following format:

$$\mathbf{Reply}_i^j = \{(\sigma_d, b_d^j, t_d^j), \forall (\sigma_d, \pi_v, T_m) \in \mathbf{Request}_j^i\}, \tag{5.10}$$

where $\sigma_d$ is the requested assisting action by robot $i$; $b_d^j$ is a boolean variable indicating the feasibility of robot $j$ offering action $\sigma_d$ at region $\pi_v$; $t_d^j \geq 0$ is time when that can happen. We describe how to determine $b_d^j$ and $t_d^j$ below.

Denote by $\overline{T}_j \geq 0$ the finishing time of the current collaboration robot $i$ is engaged in. It is initialized as 0 and updated in Section 5.2.3. As shown in Algorithm 5.8, (I) if $\overline{T}_j > 0$, it means robot $j$ is engaged in a collaboration. Then $\mathbf{Reply}_i^j = \{(\sigma_d, \perp, 0), \forall (\sigma_d, \pi_v, T_m) \in \mathbf{Request}_j^i\}$, meaning that robot $j$ would *reject* any request before its current collaboration is finished. (II) If $\overline{T}_j = 0$, it means robot $j$ is available to offer assisting actions. Then for each request $(\sigma_d, \pi_v, T_m) \in \mathbf{Request}_j^i$, robot $i$ needs to evaluate it in terms of feasibility and cost to determine $b_d^j$ and $t_d^j$. Clearly, robot $j$ needs to potentially revise its current plan to incorporate the request, i.e., to offer the assisting action $\sigma_d$ at region $\pi_v$ by time $T_m$. Denote by $\tau_{\mathcal{R},t^-}^j$ the plan of robot $j$ before the *potential* revision, of which the corresponding accepting run is $R_{p,t^-}^j$. Assume that robot $j$'s current state $q_{p,t}^j$ is the $l_{th}$ element of $R_{p,t^-}^j$ and the accepting state $q_{p,f}^j$ is the last and $f_{th}$ element. Then the segment from $q_{p,t}^j$ to

**Algorithm 5.9** Evaluate the request, `EvalReq`$(\cdot)$

**Input**: $\widehat{R}^j_{p,-}$, $(\pi_v, \sigma_d, T_m)$, $q^j_{p,t}$, $\mathcal{A}_{p,j}$
**Output**: $\widehat{R}^j_{p,+}$, $b^j_d$, $t^j_d$
$q^j_{p,t} = \widehat{R}^j_{p,-}[1]$, $q^j_{p,f} = \widehat{R}^j_{p,-}[\text{-}1]$
Compute $S_d$, $S_c$
$\bar{c} = \texttt{Cost}(\widehat{R}^j_{p,-}, \mathcal{A}_{p,j})$
$(P_1, C_1) = \texttt{DijksTA}\big(\mathcal{A}_{p,j}, q^j_{p,t}, S_d, S_c\big)$
$(P_2, C_2) = \texttt{DijksTA}\big(\texttt{Reverse}(\mathcal{A}_{p,j}), q^j_{p,f}, S_d, \varnothing\big)$
**forall the** $q^j_{p,r} \in S_d$ **do**
  **if** $P1(q^j_{p,r})$ *and* $P2(q_{p,r})$ *exist* **then**
    $C_3(q^j_{p,r}) = |C_1(q^j_{p,r}) - T_m| + \alpha_j\big(C_1(q^j_{p,r}) + C_2(q^j_{p,r}) - \bar{c}\big)$
Find the $q^{j,\star}_{p,r} \in S_d$ that minimizes $C_3(q^j_{p,r})$
**if** $q^{j,\star}_{p,r} \neq \varnothing$ **then**
  $P = P_1(q^{j,\star}_{p,r}) + \texttt{Reverse}\big(P_2(q^{j,\star}_{p,r})\big)$
  **Return** $\widehat{R}^j_{p,+} = P$, $b^j_d = \top$, $t^j_d = C_1(q^{j,\star}_{p,r})$
**Return** $\widehat{R}^j_{p,+} = \varnothing$, $b^j_d = \bot$, $t^j_d = 0$

$q^j_{p,f}$ is given by $\widehat{R}^j_{p,-} = R^j_{p,t^-}[l:f]$. We intend to find another segment $\widehat{R}^j_{p,+}$ within $\mathcal{A}_{p,j}$ from $q^j_{p,t}$ to $q^j_{p,f}$, such that by following $\widehat{R}^j_{p,+}$: (i) robot $j$ should *reach* state $\langle \pi_v, \sigma_d \rangle$; (ii) the time to reach $\langle \pi_v, \sigma_d \rangle$ should be *close* to $T_m$; (iii) the additional cost of $\widehat{R}^j_{p,+}$ compared to $\widehat{R}^j_{p,-}$ should be *small*. We enforce those conditions below.

Firstly, the set of product states in $\mathcal{A}_{p,j}$ corresponding to $\langle \pi_v, \sigma_d \rangle$ is given by $S_d = \{q_p \in Q^j_p \,|\, q_p|_{\Pi^j_{\mathcal{R}}} = \langle \pi_v, \sigma_d \rangle\}$. Consider $\widehat{R}^j_{p,+}$ with the following structure:

$$\widehat{R}^j_{p,+} = q^j_{p,t} \cdots q^j_{p,r} \cdots q^j_{p,f}, \tag{5.11}$$

where $q^j_{p,r} \in S_d$, meaning that it passes through at least one state within $S_d$. Thus the corresponding plan would contain $\langle \pi_v, \sigma_d \rangle$, which fulfils the condition (i) above. Regarding conditions (ii) and (iii), we define the *balanced* cost of $\widehat{R}^j_{p,+}$:

$$\begin{aligned} & \texttt{BalCost}(\widehat{R}^j_{p,+}, T_m, \mathcal{A}_{p,j}) \\ = \; & |\sum_{s=1}^{r-t} W^j_p\big(\widehat{R}^j_{p,+}[s], \widehat{R}^j_{p,+}[s+1]\big) - T_m| \\ & + \alpha_j \left(\texttt{Cost}(\widehat{R}^j_{p,+}, \mathcal{A}_{p,j}) - \texttt{Cost}(\widehat{R}^j_{p,-}, \mathcal{A}_{p,j})\right), \end{aligned} \tag{5.12}$$

where the first part stands for the time gap between the requested time $T_m$ by robot $i$ and the actual time based on $\widehat{R}^j_{p,+}$ (for condition (ii)); the second term is the additional cost of $\widehat{R}^j_{p,+}$, compared to $\widehat{R}^j_{p,-}$ (for condition (iii)); $\alpha_j > 0$ is a design parameter as the relative weighting.

**Problem 5.4.** *Given $\widehat{R}^j_{p,-}$, $S_d$ and $\mathcal{A}_{p,j}$, find the path segement $\widehat{R}^j_{p,+}$ that minimizes* (5.12). ▲

Algorithm 5.9 solves the above problem by the *bidirectional* Dijkstra algorithm. It utilizes the function $\texttt{DijksTA}(\cdot)$ that computes shortest paths in a weighted graph from the single source state to every state in the set of target states, while at the same time *avoiding* a set of states. It is a simple extension of the classic Dijkstra shortest path algorithm [97]. $\texttt{DijksTA}(\mathcal{A}_{p,j}, q^j_{p,t}, S_d, S_c)$ determines the shortest path (saved in $P_1$) from $q^j_{p,t}$ to every state in $S_d$ while avoiding any state belonging to $S_c$ and the associated costs (saved in $C_1$), where $S_c$ is the set of all product states associated with a collaborative *or* an assisting action: $S_c = \{q_p \in Q_{p,j} \,|\, q_p|_{\Pi^j_{\mathcal{R}}} = \langle \pi^j_{\mathcal{R}}, \sigma_n \rangle, \sigma_n \in \Sigma^j_c \cup \Sigma^j_h\}$. Then $\texttt{DijksTA}(\texttt{Reverse}(\mathcal{A}_{p,j}), q^j_{p,f}, S_d, \varnothing)$ is called to determined the shortest path (saved in $P_2$) from $q^j_{p,f}$ to every state in $S_d$ within the reversed $\mathcal{A}_{p,j}$ and the associated distances (saved in $C_2$); $\texttt{Reverse}(\mathcal{A}_{p,j})$ is the directed graph obtained by inverting the direction of all edges in $G(\mathcal{A}_{p,j})$ while keeping the weights unchanged, where $G(\mathcal{A}_{p,j})$ is the directed graph associated with $\mathcal{A}_{p,j}$ [11]. For each state $q^j_{p,r} \in S_d$, the balanced cost of the corresponding $\widehat{R}^j_{p,+}$ by (5.12) is computed. The one that yields the minimal cost is denoted by $q^{j,\star}_{p,r}$. At last, $\widehat{R}^j_{p,+}$ is formed by concatenating the shortest path from $q^j_{p,t}$ to $q^{j,\star}_{p,r}$ and the reversed shortest path from $q^j_{p,f}$ to $q^{j,\star}_{p,r}$. Last but not least, if $q^{j,\star}_{p,r}$ returns empty, it means that robot $i$ could not offer the requested collaboration thus $b^j_d = \perp$ and $t^j_d = 0$. The complexity [89, 97] of function $\texttt{DijksTA}(\cdot)$ over $\mathcal{A}_{p,j}$ is $\mathcal{O}(|\delta_{p,j}| \cdot \log |Q_{p,j}|)$; reversing $\mathcal{A}_{p,j}$ has the complexity linear to $\mathcal{O}(|\delta_{p,j}|)$, where $|Q_{p,j}|$, $|\delta_{p,j}|$ are the number of states and transitions in $\mathcal{A}_{p,j}$.

***Remark* 5.6.** Note that $\widehat{R}^j_{p,+}$ is the *potentially-revised* run, i.e., robot $j$ does not change its current plan but saves $\widehat{R}^j_{p,+}$ in $\widehat{P}$ (see Algorithm 5.8) and waits for the confirmation from robot $i$, which will be discussed in Section 5.2.3. ▲

It is worth mentioning that in case robot $i$ receives requests from multiple robots, it needs to reply to one robot first and wait for the confirmation before it replies to the next robot. Table 5.2 shows the request and reply messages regarding two collaborative actions in Section 5.3.

**Lemma 5.1.** *If $b^j_d = \top$ from Algorithm 5.9, action $\sigma_d$ can be done at the time $t^j_d$ by robot $j$ following $\widehat{R}^j_{p,+}$.*

*Proof.* Since the first segment of $\widehat{R}^j_{p,+}$ from $q^j_{p,t}$ to $q^{j,\star}_{p,r}$ is derived by $\texttt{DijksTA}(\cdot)$ of Algorithm 5.9, it does not contain any collaborative or assisting actions except $\sigma_d$. Thus it can be accomplished by robot $j$ itself with only motions and local actions, of which the time is $t^j_d$. ■

| **Request** | $\mathfrak{R}_1, (h_B, r_4, 11)$ | $\mathfrak{R}_4, (h_{C_1}, r_5, 14)$ | $\mathfrak{R}_4, (h_{C_2}, r_5, 14)$ |
|---|---|---|---|
| $\mathfrak{R}_1$ | $--$ | $(h_{C_1}, \perp, 0)$ | $(h_{C_2}, \perp, 0)$ |
| $\mathfrak{R}_2$ | $(h_B, \top, 13.1)$ | $(h_{C_1}, \top, 14.6)$ | $(h_{C_2}, \perp, 0)$ |
| $\mathfrak{R}_3$ | $(h_B, \perp, 0)$ | $(h_{C_1}, \perp, 0)$ | $(h_{C_2}, \top, 16.2)$ |
| $\mathfrak{R}_4$ | $(h_B, \perp, 0)$ | $--$ | $--$ |
| $\mathfrak{R}_5$ | $(h_B, \top, 15.7)$ | $(h_{C_1}, \top, 15.4)$ | $(h_{C_2}, \top, 15.4)$ |
| $\mathfrak{R}_6$ | $(h_B, \top, 18.2)$ | $(h_{C_1}, \perp, 0)$ | $(h_{C_2}, \perp, 0)$ |

Table 5.2: Request and reply messages exchanged for collaborative actions $o_M$ and $a_C$ in Section 5.3, regarding assisting actions $h_B$ and $h_{C_1}$, $h_{C_2}$.

### Confirmation

Based on the replies from $j \in \mathcal{N}_i$, robot $i$ needs to acknowledge them by sending back confirmation messages:

$$\mathbf{Confirm}_j^i = \{(\sigma_d,\, c_d^j,\, f_m), \forall \sigma_d \in \texttt{Depd}_i(\sigma_m)\}, \tag{5.13}$$

where $\sigma_d$ is the requested assisting action; $c_d^j$ is a boolean variable, indicating whether robot $j$ is confirmed to provide $\sigma_d$; $f_m$ is the time to finish action $\sigma_m$. The choices of $\{c_d^j, j \in \mathcal{N}_i\}$ should satisfy two constraints: (i) *exactly* one robot in $\mathcal{N}_i$ can be the confirmed collaborator for each action $\sigma_d \in \texttt{Depd}_i(\sigma_m)$; (ii) each robot in $\mathcal{N}_i$ can be confirmed for *at most* one action in $\texttt{Depd}_i(\sigma_m)$. Meanwhile, the finishing time $f_m$ should be as early as possible.

Let $|\mathcal{N}_i| = N_1$ and $|\texttt{Depd}_i(\sigma_m)| = N_2$. Without loss of generality, denote by $\mathcal{N}_i = \{1, \cdots, N_1\}$ and $\texttt{Depd}_i(\sigma_m) = \{\sigma_1, \cdots, \sigma_{N_2}\}$. The problem of finding $\{c_d^j\}$ and $f_m$ can be readily formulated as an integer programming problem [152]:

$$\begin{aligned}
&\mathbf{min} \quad f_m \\
&\mathbf{s.t.} \quad f_m = \mathbf{max}_d\{c_d^j \cdot t_d^j,\, T_m\} \\
&\qquad \sum_{d=1}^{N_2} b_d^j \cdot c_d^j \le 1, \quad \forall j \in \{1, \cdots, N_1\}, \\
&\qquad \sum_{j=1}^{N_1} b_d^j \cdot c_d^j = 1, \quad \forall d \in \{1, \cdots, N_2\},
\end{aligned} \tag{5.14}$$

where $(\sigma_d,\, b_d^j,\, t_d^j) \in \mathbf{Reply}_i^j$ from (5.10). Any stand-alone integer programming solver can be used to obtain $\{c_d^j\}$ and $f_m$ once (5.14) is formulated, e.g., "`Gurobi`" and "`CVXOPT`". Then $\forall j \in \mathcal{N}_i$ and $\forall \sigma_d \in \texttt{Depd}_i(\sigma_m)$, consider two cases: (I) if (5.14) has a solution, both $\{c_d^j\}$ and $f_m$ exist. If $c_d^j$ is $\top$, add $(\sigma_d,\, \top,\, f_m)$ to $\mathbf{Confirm}_j^i$; otherwise, add $(\sigma_d,\, \perp,\, 0)$ to $\mathbf{Confirm}_j^i$; (II) if (5.14) has no solutions, add $(\sigma_d,\, \perp,\, 0)$ to $\mathbf{Confirm}_j^i$. It means that $\sigma_m$ can not be fulfilled according to the current replies. Then how robot $i$ needs to delay $\sigma_m$ and revise its plan will be given in Section 5.2.3.

**Algorithm 5.10** Delay collaboration, `DelayCol()`

**Input**: $\widehat{R}^i_{p,-}$, $q^i_{p,t}$, $\langle \pi_v, \sigma_d \rangle$, $\mathcal{A}_{p,i}$, $\widehat{\Sigma}^i_c$
**Output**: $\widehat{R}^i_{p,+}$
Compute $S_d$ given $\langle \pi_v, \sigma_d \rangle$, $S_c$
$q^i_{p,t} = \widehat{R}^i_{p,-}[1]$, $q^i_{p,f} = \widehat{R}^i_{p,-}[-1]$
$(P_1, C_1) = \texttt{DijksTA}\big(\mathcal{A}_{p,i}, q^i_{p,t}, S_d, S_c\big)$
$(P_2, C_2) = \texttt{DijksTA}\big(\texttt{Reverse}(\mathcal{A}_{p,i}), q^i_{p,f}, S_d, \varnothing\big)$
**forall the** $q^i_{p,d} \in S_d$ **do**
  **if** $C_1(q^i_{p,d}) > T_m + D_i$ *and* $P_2(q^i_{p,d})$ *exists* **then**
    $\widehat{R}^k_{p,+} = P_1(q^i_{p,d}) + \texttt{Reverse}\big(P_2(q^i_{p,d})\big)$
    **Return** $\widehat{R}^i_{p,+}$

***Remark* 5.7.** The optimization problem (5.14) is solved locally by robot $i$ regarding the requested collaborative task $\sigma_m$, with $|\mathcal{N}_i| \cdot |\texttt{Depd}_i(\sigma_m)|$ Boolean variables. ▲

### Plan adaptation

After sending out the confirmation messages, robot $i$ checks the following: (I) if (5.14) has a solution, it means that $\sigma_m$ can be fulfilled and $R^i_{p,t}$ remains unchanged. $\overline{T}_i$ is set to $f_m$ to indicate that robot $i$ is engaged in the collaboration until the time $f_m$; (II) otherwise, it means that according to the current replies $\sigma_m$ can not be done as planned in $R^i_{p,t}$. Thus robot $i$ needs to revise its plan by delaying this collaborative action $\sigma_m$. Algorithm 5.10 revises $R^i_{p,-}$ and delays $\sigma_m$ by time $D_i$, where $D_i > 0$ is a design parameter. Function $\texttt{DijksTA}(\cdot)$ from Algorithm 5.9 is used to find a path from $q^i_{p,t}$ to one state in $S_d$ whose cost is larger than $T_m + D_i$ and at the same time reachable to the accepting state $q^i_{p,f}$. Such a path can always be found as the action $\sigma_0$ that takes time $T_0$ can be repeated as many times as needed.

On the other hand, upon receiving $\mathbf{Confirm}^i_j$, each robot $j \in \mathcal{N}_i$ checks the following: (I) if $b^j_d = \top$, it means robot $j$ is confirmed to offer the assisting action $\sigma_d$. As a result, it modifies its plan based on the potential set of plans $\widehat{P}$ from Algorithm 5.9. In particular, the plan segment $\widehat{R}^j_p$ is set to $\widehat{R}^j_{p,+}$ and $\overline{T}_j$ is set to $f_m$; (II) if robot $b^j_d = \bot, \forall \sigma_d \in \texttt{Depd}_i(\sigma_m)$, it means robot $j$ is not confirmed as a collaborator. Then $R^j_p$ remains unchanged and $\overline{T}_j$ is set to 0. Afterwards, robot $i$ and all confirmed collaborators in $\mathcal{N}_i$ would execute its plan by following the motion and action in sequence. They would reject any further request as described in Section 5.2.3 until the collaboration for action $\sigma_m$ is done.

**Theorem 5.2.** *If* (5.14) *has a solution, the time of accomplishing action* $\sigma_m$ *is* $f_m$.

*Proof.* Since each assisting action $\sigma_d \in \texttt{Depd}_i(\sigma_m)$ has been assigned exactly to one robot $j \in \mathcal{N}_i$, then $\sigma_d$ can be accomplished by robot $j$ at time $t^j_d$ by Lemma 5.1.

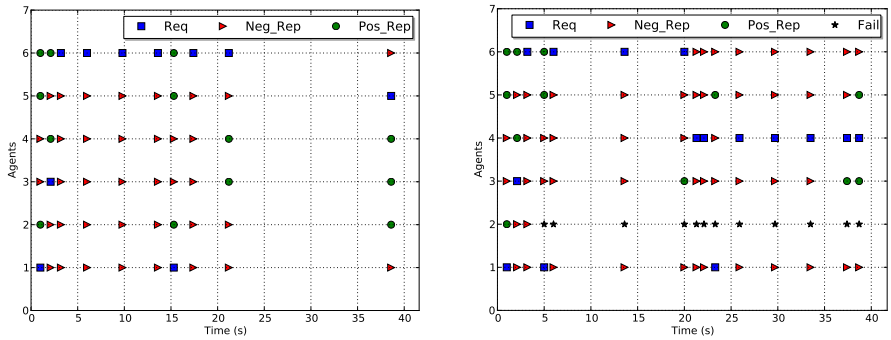


Figure 5.2: Messages exchanged for both scenarios of Section 5.3. Blue square stands for request messages by (5.9) while red triangles and green dots stand for reply messages by (5.10) with $b_d^j$ being $\perp$ and $\top$. Black stars indicate the robot failure.

Thus $\sigma_m$ can be accomplished by robot $i$ at the time $f_m$, which is the latest time for all actions by (5.14). ■

Since each robot has a finite plan as a finite sequence of motion and actions, when one robot finishes executing its plan, it would become an assisting robot by setting its task $\varphi_i \triangleq \top$. Then it would stay at one region unless it is confirmed to collaborate with others.

### 5.2.4 Failure detection and recovery

Due to the unavoidable characteristics and constraints of physical robots, any robot may fail (stop being functional) at anytime. Detection of this type of failure is particularly important for collaborations involving several robots.

We propose an inquiry and acknowledgment mechanism as follows: robot $i$ that has confirmed on its collaborative action $\sigma_m$ would continuously monitor the status of its confirmed collaborators for each assisting action $\sigma_d \in \mathtt{Depd}_i(\sigma_m)$ until $\sigma_m$ is done. If, for instance, robot $h \in \mathcal{N}_i$ who is confirmed to offer action $\sigma_d$ fails to acknowledge robot $i$'s inquiry for a limited time, robot $i$ may assume that robot $h$ has failed. As a result, robot $i$ needs to re-assign the assisting action $\sigma_d$ to another robot in $\mathcal{N}_i$, as follows: (i) send a new request $\{(\pi_v, \sigma_d, T_m)\}$ to all neighbors within $\mathcal{N}_i$ and wait for the reply; (ii) based on the replies $\{(\sigma_d, b_d^j, t_d^j)\}, \forall j \in \mathcal{N}_i\}$, choose the new collaborator $\hat{h} \in \mathcal{N}_i$ for action $\sigma_d$ as follows: $\hat{h} = \operatorname{argmin}_{j \in \mathcal{N}_i} \{|f_m - t_d^j \cdot b_d^j|\}$, where $f_m$ is the previously confirmed time from (5.14). Namely, it chooses robot $\hat{h}$ that can offer action $\sigma_d$ at the time closest to $f_m$; (iii) send the confirmation $\{(\sigma_d, \top, f_m)\}$ to robot $\hat{h}$; (iv) robot $\hat{h}$ adapts its plan and be engaged in the collaboration on $\sigma_m$ as described in Section 5.2.3.

**Loosely-coupled system**

As mentioned earlier, we aim to apply this distributed coordination scheme to loosely-coupled multi-robot systems, where collaborations among the robots are (i) local in the sense that only neighboring robots are needed; (ii) sparse in the sense that they are needed infrequently compared with the total number of activities of all robots required by their local tasks, which imposes the following assumption:

**Assumption 5.1.** *There exists a finite time* $\mathbf{T} > 0$ *such that for each robot* $i \in \mathcal{N}$ *and any collaborative action* $\sigma_m$ *requested by robot* $i$ *initially at time* $t_m > 0$*, problem* (5.14) *for* $\sigma_m$ *will have a solution within time* $t_m + \mathbf{T}$*.* ▲

Namely, the above assumption indicates that any collaborative action required by each robot $i \in \mathcal{N}$ in order to satisfy the local task $\varphi_i$ should always eventually be provided in finite time by robot $i$ and its neighbors.

### 5.2.5 Overall structure

During the real-time execution, each robot executes its plan and checks first if any request is received. If so, it replies to them by Algorithm 5.8, waits for the confirmation and adjusts its plan accordingly. Otherwise, it sends out requests by Algorithm 5.7, waits for reply, sends confirmation back by (5.14) and at last adapts its plan by Algorithm 5.10. The correctness of the proposed scheme is guaranteed by Theorem 5.3 below:

**Theorem 5.3.** *Under Assumption 5.1, the proposed coordination scheme solves Problem 5.2. Namely, all local tasks* $\varphi_i$ *can be accomplished in finite time,* $\forall i \in \mathcal{N}$*.*

*Proof.* Starting from the initial plan $\tau^i_{\mathcal{R},\text{init}}$ for robot $i \in \mathcal{N}$, motion and local actions in $\tau^i_{\mathcal{R},\text{init}}$ can be accomplished locally by an robot itself. If $\tau^i_{\mathcal{R},\text{init}}$ remains unchanged for robot $i$, we only need to show that collaborative actions in $\tau^i_{\mathcal{R},\text{init}}$ can be accomplished. The fact that $\tau^i_{\mathcal{R},\text{init}}$ remains unchanged indicates that robot $i$'s requests for each collaborative action $\sigma_m$ are fulfilled, i.e., (5.14) for $\sigma_m$ has a solution. By Theorem 5.2, action $\sigma_m$ can be accomplished in finite time $f_m$. Since $\tau^i_{\mathcal{R},\text{init}}$ is a finite sequence, $\varphi_i$ can be satisfied in finite time as every motion and action inside can be done in finite time. On the other hand, if $\tau^i_{\mathcal{R},\text{init}}$ has to be adapted in real-time, the reasons are: (i) robot $i$ is confirmed to assist its neighbor $j$ on one collaboration; (ii) robot $i$ has made a request for a collaborative action $\sigma_m$ and it is delayed by Algorithm 5.10 as (5.14) has no solution. For case (i), Algorithm 5.9 guarantees that after the assistance its updated run $\hat{R}^i_{p,+}$ still satisfies $\varphi_i$ in finite time. For case (ii), by Assumption 5.1, (5.14) will have a feasible solution within at most time $\mathbf{T}$, meaning that $\sigma_m$ will be done within finite time. ■

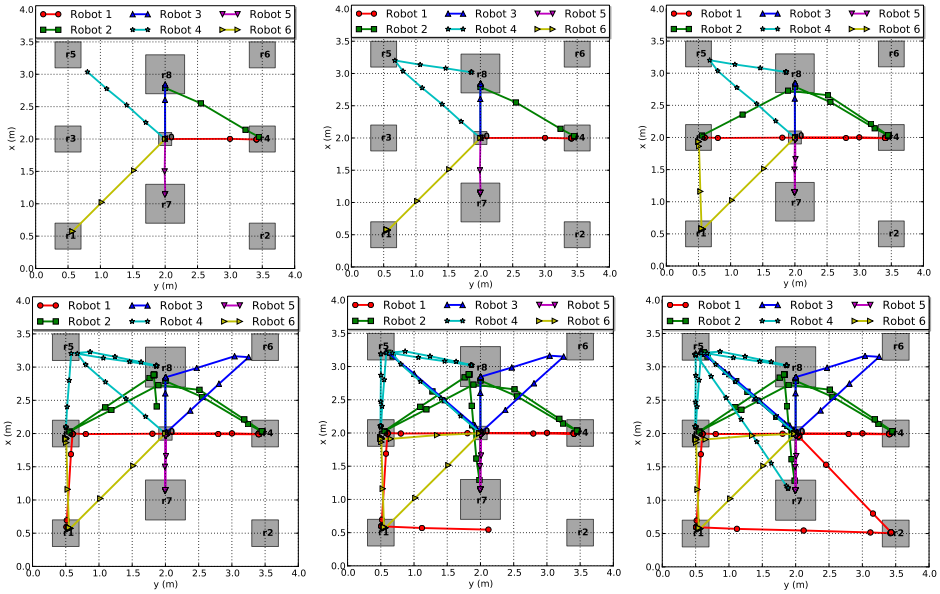


Figure 5.3: Snapshots of collaborations on $l_B$, $o_M$, $u_B$, $c_F$ and $a_C$ in Section 5.3.

## 5.3 Case study

In the case study, we present a simulated example of six autonomous robots with heterogeneous capacities. The proposed algorithms are implemented in Python 2.7.

### System description

The workspace of size $4m \times 4m$ is given in Figure 5.3, within which there are nine rectangular regions of interest $r_0$, $r_1$, $r_2, \cdots, r_8$ (in gray). The regions are labelled by the objects of interest, like A, B, C, M, E, F. Denote by the six robots $\mathfrak{R}_1, \mathfrak{R}_2, \cdots, \mathfrak{R}_6$. They all satisfy the single-integrator dynamics, i.e., $\dot{x} = u$, where $x, u \in \mathbb{R}^2$ are the 2-D position and velocity. The robots have velocities between $0.6m/s$ and $1m/s$. Besides, the action sets of each robot are shown in Table 5.1. Robot $\mathfrak{R}_1$ can load and unload ($l_A$, $u_A$) a light object A; load and unload ($l_B$, $u_B$) a heavy object B with others' assisting action $h_B$. Robot $\mathfrak{R}_2$ can take pictures ($s$); help load and unload ($h_B$) B; help assemble ($h_{C1}$) C; help charge ($h_F$) F. Robot $\mathfrak{R}_3$ can operate ($o_M$) machine M with assisting action $h_M$; help assemble ($h_{C2}$) C; help charge ($h_F$) F. Robot $\mathfrak{R}_4$ can assemble ($a_C$) C with assisting actions $h_{C1}$, $h_{C2}$; help operate ($h_M$) M; help charge ($h_F$) F. Robot $\mathfrak{R}_5$ can maintain D, help assemble ($h_{C1}$, $h_{C2}$) C, and help load and unload ($h_B$) B. Robot $\mathfrak{R}_6$ can operate object ($o_E$) E, charge object ($c_F$) F with assisting action $h_F$, help operate ($h_M$) M and help load and unload ($h_B$) B. We assume all actions have duration $10s$ and the time horizon $H_i$ is set to $20s$ uniformly. Any two robots can exchange messages directly. "`Gurobi`" for

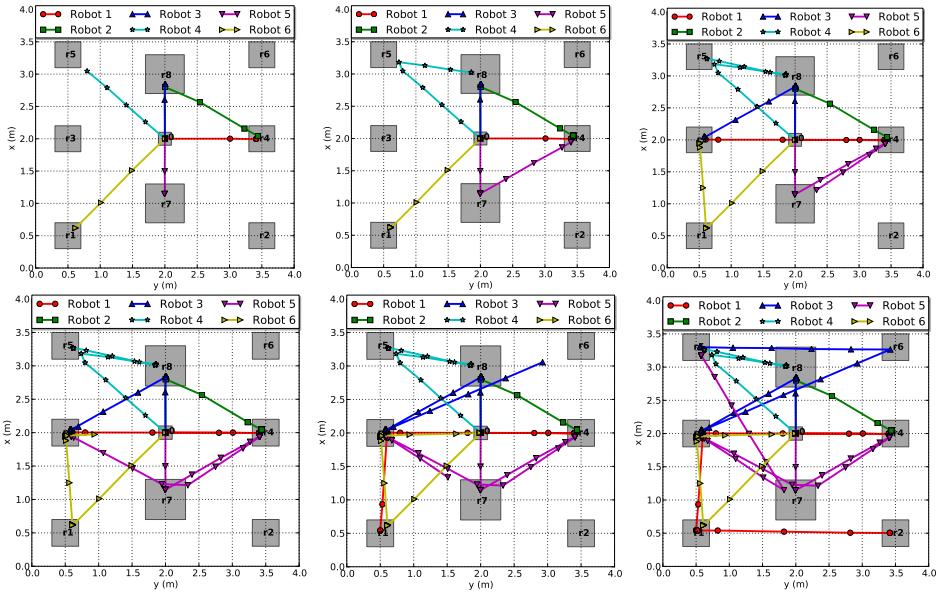


Figure 5.4: Snapshots of collaborations on $l_B$, $o_M$, $u_B$, $c_F$ and $a_C$ for Section 5.3. Note that $\mathfrak{R}_2$ failed at $t = 5s$, instead $\mathfrak{R}_5$ is re-assigned to assist $\mathfrak{R}_1$ on $l_B$, $u_B$.

Python is used as the mix integer programming solver.

Each robot is assigned a local task as follows: robot $\mathfrak{R}_1$ has to deliver A to $r_2$ and B to $r_3$. Then $\varphi_1 = \Diamond\big(l_A \wedge \Diamond(r_2 \wedge \bigcirc u_A)\big) \wedge \Diamond\big(l_B \wedge \Diamond(r_3 \wedge \bigcirc u_B)\big)$; Robot $\mathfrak{R}_2$ has to surveil regions $r_7$, $r_8$ and take pictures there. $\varphi_2 = \Diamond(r_7 \wedge \bigcirc s) \wedge \Diamond(r_8 \wedge \bigcirc s)$; Robot $\mathfrak{R}_3$ has to operate M and visit $r_6$. $\varphi_3 = \Diamond(o_M \wedge \Diamond r_6)$; Robot $\mathfrak{R}_4$ has to take a photo in $r_7$, assemble C and visit $r_6$. $\varphi_4 = \Diamond(r_7 \wedge \bigcirc s) \wedge \Diamond(a_C)$; Robot $\mathfrak{R}_5$ needs to maintain D and back to region $r_0$. $\varphi_5 = \Diamond(m_D \wedge \Diamond r_0)$; Robot $\mathfrak{R}_6$ needs to operate E first and then charge F. $\varphi_6 = \bigcirc(o_E \wedge \Diamond c_F)$.

### Simulation results

All robots start form the center $(2m, 2m)$. The synthesized initial plan of each robot is as follows: $r_0 r_4\, l_B\, r_3\, u_B\, r_1\, l_A\, r_2\, u_A$ for $\mathfrak{R}_1$; $r_0\, r_8\, s\, r_7\, s$ for $\mathfrak{R}_2$; $r_0\, r_8\, o_M\, r_6$ for $\mathfrak{R}_3$; $r_0\, r_5\, a_C\, r_7\, s$ for $\mathfrak{R}_4$; $r_0\, r_7\, m_D$ for $\mathfrak{R}_5$; $r_0\, r_1\, o_E\, r_3 c_F$ for $\mathfrak{R}_6$. We simulate first one nominal scenario and another with $\mathfrak{R}_2$'s failure since $t = 5s$. The messages exchanged during both scenarios are shown in Figure 5.2.

**Nominal scenario**. The system is simulated for $70.3s$ before all robots accomplish their tasks, during which five collaborations among the robots are accomplished. As shown in Figure 5.3, robot $\mathfrak{R}_1$ firstly sends the request on action $h_B$ and the reply messages are shown in Table 5.2. $\mathfrak{R}_2$ is confirmed as the collaborator and changes its plan to $r_0\, r_4\, h_B\, r_8\, s\, r_7\, s$. $\mathfrak{R}_1$ finishes action $l_B$ at $t = 14.3s$ At the same time, $\mathfrak{R}_4$ is chosen to help $\mathfrak{R}_3$ on action $o_M$, which is done at $t = 21.1s$. Afterwards, $\mathfrak{R}_1$

finishes $u_B$ at $t = 28.4s$ with $\mathfrak{R}_2$ offering $h_B$. Robot $\mathfrak{R}_6$'s request for action $c_F$ keeps getting delayed until $t = 21.1s$, as $\mathfrak{R}_2$, $\mathfrak{R}_3$, $\mathfrak{R}_4$ are engaged in other collaborations. Afterwards robot $\mathfrak{R}_4$ is confirmed to offer action $h_F$ and $c_F$ is done at $t = 38.0s$. After that, robot $\mathfrak{R}_4$ sends the request for $a_C$ regarding $h_{C_1}$ and $h_{C_2}$. The reply messages are shown in Table 5.2. After solving (5.14), $\mathfrak{R}_2$ and $\mathfrak{R}_3$ offer actions $h_{C_1}$ and $h_{C_2}$, which are done at $t = 54.1s$. The simulation video is online [54].

**Failure recovery**. To illustrate the effectiveness of our approach for handling robot failures, we stop simulating robot $\mathfrak{R}_2$ since $t = 5s$ whereas the other robots remain the same. As shown in Figure 5.4, initially robot $\mathfrak{R}_2$ is confirmed to offer $h_B$ to $\mathfrak{R}_1$. However since $\mathfrak{R}_2$ fails at $t = 5s$, $\mathfrak{R}_1$ detects that by the inquiry and acknowledgment mechanism described in Section 5.2.4 and resends a request regarding $h_B$, for which $\mathfrak{R}_5$ is confirmed as the new collaborator. Then $l_B$ is done at $t = 22.3s$. Afterwards, again with the help of $\mathfrak{R}_5$, $\mathfrak{R}_1$ finishes $u_B$ at $t = 37.8s$. $\mathfrak{R}_3$ finishes $o_M$ with the help of $\mathfrak{R}_4$ at $t = 29.7s$. Before this time, robot $\mathfrak{R}_6$ has to delay its action $c_F$ as both $\mathfrak{R}_3$ and $\mathfrak{R}_4$ are engaged in $o_M$ and $\mathfrak{R}_2$ has failed. Then $\mathfrak{R}_3$ offers $h_F$ to $\mathfrak{R}_6$ at $t = 36s$. After that, $\mathfrak{R}_4$ finishes $a_C$ with the help of $\mathfrak{R}_3$ and $\mathfrak{R}_5$ at $t = 68.9s$. At last, each robot fulfills its local task by $t = 76.5s$. The simulation video is online [55].

## 5.4 Summary

In this chapter, we first presented an automated framework for a single robot to fulfill its local task given as the desired motion and actions. Then we introduced a novel method to define the dependency relations within multi-robot systems using collaborative and assisting actions. Based on this dependency, a bottom-up motion and task coordination scheme was proposed where the collaborative sub-tasks are coordinated and executed successfully during run time.